

DISEÑO DE FRAGMENTACIÓN

PLANTEAMIENTO DEL PROBLEMA

Para resolver el ejercicio propuesto, el cual requiere distribuir la cartera de clientes en nodos geográficos para su administración autónoma, se determinó que la fragmentación horizontal primaria debe realizarse sobre la tabla propietaria **Sales.Customer**. En lugar de utilizar rangos arbitrarios de **CustomerID**, se seleccionó el atributo **TerritoryID**, ya que este representa fielmente la localidad geográfica real del negocio y permite cumplir con el requerimiento de autonomía regional.

Para comenzar el análisis de la fragmentación, es importante tener en consideración el grafo relacional.

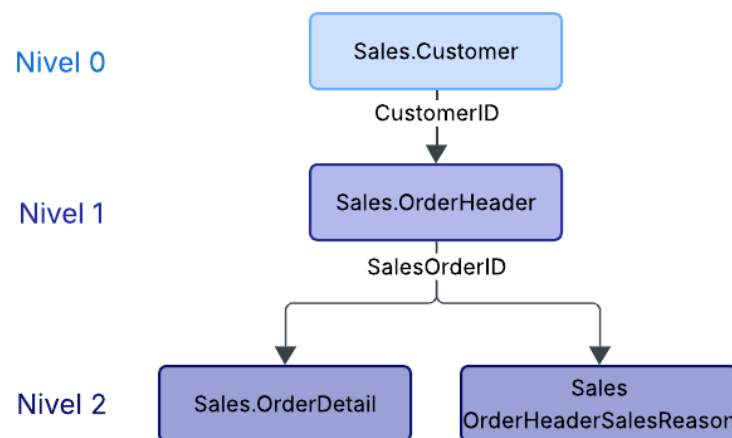


Figura 1. Grafo Relacional de la tabla Sales.Customer

En la Figura 1 podemos ver las tablas que se relacionan con nuestra tabla propietaria **Sales.Customer** directamente es **Sales.OrderHeader** siendo una tabla miembro y propietaria y en nuestro último nivel las tablas **Sales.OrderDetail** y **Sales.OrderHeaderSalesReason** las cuales son miembro de nuestra tabla **Sales.OrderHeader**.

ANÁLISIS DE DISTRIBUCIÓN Y BALANCEO DE CARGA

Al analizar la distribución actual de los datos agrupados geográficamente, se identificaron tres macrorregiones con la siguiente carga de registros:

- **North America:** 16,108 registros
- **Europe:** 8,514 registros
- **Pacific:** 6,843 registros

El análisis revela un desbalance significativo, donde Norteamérica concentra más de la mitad de las operaciones. Para garantizar un correcto balanceo de carga en una arquitectura de 3 nodos geográficos, se procedió a desglosar los territorios específicos:

TerritoryID	Región	Cantidad de Registros
1	North America	4,594
2	North America	352
3	North America	385
4	North America	6,224
5	North America	486
6	North America	4,067
7	Europe	2,672
8	Europe	2,623
9	Pacific	6,843
10	Europe	3,219

Tabla 1. Consulta de registros por Territorio

Grupo Territorio	Cantidad de registros
North America	16108
Europe	8514
Pacific	6843

Tabla 2. Consulta de registros por Grupo Territorio

Con base en esta cantidad de datos, se decidió dejar Norteamérica con su registros asignados y fusionar Europa y el Pacífico, generando la siguiente distribución optimizada:

- **Nodo 1 (Norteamérica):** Territorios 1 al 6 (Carga: 16,108 registros).
- **Nodo 2 (Europa y Pacífico):** Territorios 7, 8, 9 y 10 (Carga: 15,357).

ALGORITMO COM_MIN Y MINITÉRMINOS

Para aplicar el algoritmo COM_MIN, se definen los predicados simples que delimitarán los fragmentos.

Conjunto de Predicados (Pr):

- p_1 : TerritoryID = 1
- p_2 : TerritoryID = 2
- p_3 : TerritoryID = 3
- p_4 : TerritoryID = 4
- p_5 : TerritoryID = 5
- p_6 : TerritoryID = 6
- p_7 : TerritoryID = 7
- p_8 : TerritoryID = 8
- p_9 : TerritoryID = 9
- p_{10} : TerritoryID = 10

Constituye el listado final obtenido tras ejecutar la fase de depuración mediante el algoritmo COM_MIN. Con base en los fundamentos propuestos por Özsu y Valduriez (2020), la colección de condiciones derivadas (Pr') requiere cumplir rigurosamente con los criterios de relevancia y completitud. En el esquema evaluado, se demuestra formalmente que la agrupación inicial (Pr) coincide plenamente con la depurada ($Pr = Pr'$). Dicha equivalencia responde a que cada zona geográfica (unidades 1 a 10) resulta determinante para la ejecución descentralizada de las consultas locales; por ende, la frecuencia transaccional varía sustancialmente según la ubicación de origen. Al constatar que ninguna condición produce particiones idénticas, se descarta la presencia de redundancias, garantizando la preservación íntegra del conjunto.

Minitérminos

$$\begin{aligned}
 M = \{ \\
 m_1 &= p_1 \wedge p_2 \wedge p_3 \wedge p_4 \wedge p_5 \wedge p_6 \wedge \neg p_7 \wedge \neg p_8 \wedge \neg p_9 \wedge \neg p_{10} \\
 m_2 &= \neg p_1 \wedge \neg p_2 \wedge \neg p_3 \wedge \neg p_4 \wedge \neg p_5 \wedge \neg p_6 \wedge p_7 \wedge p_8 \wedge p_9 \wedge p_{10} \\
 m_3 &= \neg p_1 \wedge \neg p_2 \wedge \neg p_3 \wedge \neg p_4 \wedge \neg p_5 \wedge \neg p_6 \wedge \neg p_7 \wedge \neg p_8 \wedge \neg p_9 \wedge \neg p_{10} \\
 \}
 \end{aligned}$$

Estos minitérminos fueron el resultado de representar los dos nodos previamente descritos. La elección de esta distribución en 3 minitérminos (y por ende, 3 nodos lógicos) se justifica por dos factores clave documentados en la literatura para el diseño de bases de datos distribuidas (Noaman & Barker, 1999): la optimización del rendimiento mediante el balanceo de carga y el control del sesgo de datos (data skew). Al agrupar lógicamente Europa y Pacífico en m_2 (15,357 registros), se logra equiparar la carga transaccional frente al fragmento dominante de Norteamérica en m_1 (16,108 registros), evitando cuellos de botella de procesamiento o almacenamiento en un solo nodo físico.

Adicionalmente, el minitérmino m_3 se diseñó estrictamente como un fragmento residual (Catch-all) para capturar excepciones (valores nulos, errores de captura o nuevos

territorios en expansión). Esto es indispensable para asegurar que el diseño abarque el dominio infinito del atributo y no se pierdan datos por fuera de los 10 territorios conocidos.

Dado que las condiciones geográficas son mutuamente excluyentes (un cliente no puede pertenecer a dos territorios simultáneamente), las combinaciones lógicas que generarían traslapes se anulan. Por lo tanto, los predicados operan directamente como nuestros **Minitérminos** (m_1 a m_3), originando los 3 fragmentos primarios de la tabla clientes. Quedaría de la siguiente manera:

$$\text{Customer}_i = \sigma_{m_i}(\text{Customer})$$

Desarrollando:

$$\text{Customer}_1 = \sigma_{m_1}(\text{Customer})$$

$$\text{Customer}_2 = \sigma_{m_2}(\text{Customer})$$

$$\text{Customer}_3 = \sigma_{m_3}(\text{Customer})$$

El fragmento i de clientes se forma seleccionando únicamente las filas de la tabla Customer que cumplen con la condición del predicado m_i .

FRAGMENTACIÓN DERIVADA (OPERACIÓN SEMI-JOIN)

Para mantener la localidad de referencia, las tablas dependientes deben ubicarse en el mismo nodo físico que su registro propietario, como se puede ver en la Figura 1 del grafo relacional.

Esta propagación se define aplicando la operación Semi-Join ($\bowtie$) para cada uno de los 3 nodos (donde $i = 1, 2, 3$):

Nivel 1: Sales Order Header (SOH)

$$\text{SOH}_i = \text{SOH} \bowtie_{\text{CustomerID}} \text{Customer}_i$$

Desglosado:

$$\text{SOH}_1 = \text{SOH} \bowtie_{\text{CustomerID}} \text{Customer}_1$$

$$\text{SOH}_2 = \text{SOH} \bowtie_{\text{CustomerID}} \text{Customer}_2$$

$$\text{SOH}_3 = \text{SOH} \bowtie_{\text{CustomerID}} \text{Customer}_3$$

Se extrae de la tabla SOH solamente las filas cuyo CustomerID exista dentro del conjunto del fragmento Customer_i .

Nivel 2: Sales Order Details (SOD) y Sales Order Header Sales Reason (SOHSR)

$$\text{SOD}_i = \text{SOD} \bowtie_{\text{SalesOrderID}} \text{SOH}_i$$

Desglosando:

$$\text{SOD}_1 = \text{SOD} \bowtie_{\text{SalesOrderID}} \text{SOH}_1$$

$$\begin{aligned} SOD_2 &= SOD \bowtie_{SalesOrderID} SOH_2 \\ SOD_3 &= SOD \bowtie_{SalesOrderID} SOH_3 \end{aligned}$$

$$SOHSR_i = SOHSR \bowtie_{SalesOrderID} SOH_i$$

Desglosando:

$$\begin{aligned} SOHSR_1 &= SOHSR \bowtie_{SalesOrderID} SOH_1 \\ SOHSR_2 &= SOHSR \bowtie_{SalesOrderID} SOH_2 \\ SOHSR_3 &= SOHSR \bowtie_{SalesOrderID} SOH_3 \end{aligned}$$

VALIDACIÓN DE LAS REGLAS DE FRAGMENTACIÓN (ÖZSU)

El diseño propuesto es matemáticamente robusto y cumple con las tres reglas fundamentales de la fragmentación distribuida:

1. **Complejitud:** La literatura estipula que cada tupla de la relación global debe ser mapeada a al menos un fragmento para evitar pérdida de datos. Todos los registros históricos tienen un nodo asignado (m1 a m2). Cualquier registro futuro o con un TerritoryID no mapeado será capturado obligatoriamente por el nodo m3, garantizando que no exista pérdida de información. Asimismo, los Semi-Joins aseguran la completitud equivalente de las tablas miembro al propagar esta regla a través de las llaves foráneas.
2. **Reconstrucción:** Debe existir una operación relacional capaz de restaurar la relación original sin pérdida de semántica. Tratándose de una fragmentación horizontal, la regla se cumple ya que la tabla centralizada original puede regenerarse íntegramente mediante la unión relacional (Union) de los fragmentos distribuidos: $Customer = Customer1 \cup Customer2 \cup Customer3$.
3. **Disyunción:** Özsu señala que los datos no deben estar replicados innecesariamente en fragmentaciones horizontales para facilitar el control de concurrencia y actualizaciones. Los predicados primarios son mutuamente excluyentes ($Customer_i \cap Customer_j = \emptyset$) ya que, operativamente, un cliente es asignado a una sola ubicación geográfica. Esta exclusividad se transfiere a las tablas dependientes gracias a la integridad referencial obligatoria.

CONCLUSIÓN

El desarrollo de esta propuesta de diseño evidenció que el reto principal en la fragmentación de bases de datos distribuidas radica en la fase de análisis inicial. Fue fundamental evaluar meticulosamente la volumetría de los datos existentes para identificar los desbalances de carga geográfica, así como rastrear cuidadosamente las dependencias referenciales a partir de la tabla propietaria para estructurar un grafo relacional preciso. Una vez superado este análisis y comprendida la necesidad operativa del negocio, la transición hacia el marco teórico de Özsu se volvió sumamente metódica. Al contar con un cimiento claro, la aplicación del algoritmo COM_MIN resultó un proceso ágil y estructurado, lo que facilitó enormemente la definición de los predicados lógicos, la generación de los minitérminos y la derivación de los fragmentos mediante operaciones semi-join. La consolidación de la carga en dos macrorregiones principales (Norteamérica vs. Europa/Pacífico), combinada con la inserción de un tercer fragmento estratégico para el manejo de excepciones, culmina en un modelo de distribución altamente robusto, escalable y que cumple de forma estricta con las reglas de completitud, reconstrucción y disyunción.

REFERENCIAS

1. Özsu, M. T., & Valduriez, P. (2020). *Principles of Distributed Database Systems* (4.^a ed.). Springer.
2. Noaman, A. Y., & Barker, K. (1999). *A horizontal fragmentation algorithm for the fact relation in a distributed data warehouse*. En *Proceedings of the eighth international conference on Information and knowledge management* (pp. 154-161). ACM.